Foundation Pose Models for SLAM

Aiden Swann
Dept. of Mechanical Engineering
Stanford University
Stanford, USA
swann@stanford.edu

Lachlain McGranahan
Dept. of Aero/Astro
Stanford University
Stanford, US
lachmcg@stanford.edu

***Abstract*—Simultaneous Localization and Mapping (SLAM) has been fundamental in robotics and computer vision, enabling many advancements in robot navigation. Traditional SLAM methods have relied heavily on geometric constraints and feature-based approaches, which can struggle in texture-less and otherwise challenging scenes. The emergence of foundation models trained on large-scale vision datasets has opened new possibilities for SLAM and visual odometry. These models demonstrate robust and generalizable performance across diverse scenes with 0-shot performance. In this project we hope to take the first steps towards the integration of foundation pose models with SLAM systems. We hope to start by benchmarking state of the art vision pose foundation models against traditional SLAM baselines and ground truth motion capture data. Pending the adequate performance of an vision pose model we will incorporate this model into a larger visual inertia odometry system. We hope to achieve stable drone flight using only the foundation pose model and onboard drone sensing. Further potential improvements could include fine-tuning the foundation pose model on our specific data or focusing on achieving high accuracy enviromenmental mapping.**

***Index Terms*—SLAM, Bundle Adjustment, NeRF, Guassian Splat, ViT**

I. Introduction

Localization and mapping is a fundamental problem for robotic systems and is the baseline for almost all other navigation algorithms. The process by which a robot preforms this localization and mapping is known as SLAM. SLAM was first used successfully for vehicle localization and was introduced into the spotlight by its successful implementation is the DARPA urban challenge in 2000 [1]. Today's SLAM algorithms have improved significantly relying on complex feature matching with graph based optimizations. [2], [3]. Large foundation models have solved problem after problem in computer vision including classification [4], feature detection [5] and depth perception [6]. Over and over large feed forward networks have replaced traditional computer vision techniques offering high performance and faster execution. Recently camera pose estimation foundation models have emerged [7]–[9]. In this project we explore the feasibility of using this models and a replacement for traditional visual SLAM on the quadrotor system.

II. Related Work

A. Traditional SLAM

Simultaneous Localization and Mapping (SLAM) is a long standing problem in robotics [1]. Early SLAM methods relied on EKF like filtering, which evolved to more complex graph optimization based approaches. In MonoSLAM [3] real time monocular slam is accomplished by tracking features in a probabilistic framework. Methods like ORBSLAM [2] significancy increased performance on monocular and RGB-D scenes through robust feature matching and extraction algorithms. Methods like COLMAP [10], [11] rely on structure from motion (SfM) to extract camera intrinsics and extrinsics and intrisics as well as sparse point clouds. While COLMAP is not suitable for real time use, it produces highly accurate camera poses from only monocular information.

While modern SLAM methods are exceptionally powerfully, enabling everything from quadrotor flight to VR headsets, they have several limitations. Firstly they struggle in challenging scenes with changing lightening conditions or fast movements. Secondly their environmental reconstruction while geometrically accurate often lacks high visual accuracy which may be useful for downstream tasks.

B. Radiance Field SLAM

Radiance fields are an environment representation method which has recently gained prominence with the introduction of neural radiance fields (NeRFs) [12]. NeRFs allow a full environment to be reconstructed from a series of images. The resulting radiance field is highly visually accurate and can be used for many downstream tasks such as navigation [13] and manipulation [14]. The method of 3D Gaussian Splatting (3DGS) [15] has recently advanced the field by providing high-quality and high-speed training along with realtime rendering.

Due to the visually and geometrically expressive nature of these representations they are a naturally choice for SLAM. Many papers have combined NeRF with SLAM [16]–[19]. These works range from applying traditional SLAM pipelines soley to pose estimation [16] to hybrid approaches [18], [19], which leverage traditional SLAM optimization with the NeRF representation. 3DGS has more successfully been adapted to be used as a SLAM tool due to its much faster rendering and training [20], [21]. These approaches directly leverage camera extrinsic optimization through the differentiable visual representation itself. While radiance field based SLAM improves visual performance over traditional SLAM methods it is still quite slow. Even running on desktop grade hardware these radiance field base SLAM methods struggle to achieve real time due to the computational complexity of the radiance field

representation. However by utilizing vision foundation models trained on pose estimation we can achieve the expressiveness of NeRF with a single feed-forward pass.

C. Vision Foundation Models

Early vision foundation models were built on convolutional neural networks (CNNs), starting with the breakthrough ImageNet victory of AlexNet in 2012 [4]. This architecture was pushed further with the introduction of deep CNNs, often referred to as visual grounded geometry (VGG) networks, which implemented 16-19 layer stacks of 3 by 3 convolutions [22]. Traditionally these early image classification networks were supervised, meaning they required labeled training data, but self-supervised CNN frameworks such as SimCLR began to push the field towards unsupervised learning [23].

The field has since migrated toward using transformer backbones, the architecture first proposed by Google Deep Mind for translation but now known as the backbone for the boom of generative language models [24]. In 2021 the Google Brain team built upon this by introducing the first Vision Transformer (ViT) architecture, which replaced convolution with global self-attention over image patches. When pre-trained on sufficiently large image datasets, the ViT could match or surpass state-of-the-art CNNs on classification and transfer learning [25]. Subsequent self-supervised variants such as DINO scaled these ideas, with the DINOv2 network from Meta pushing the architecture to foundation model scale. The largest form of the network, using the ViT-g 1.1 billion parameter backbone, was trained without labels on a curated 142 million-image dataset, producing robust features that generalized across image distributions and tasks without the need for finetuning [5]. These transformer-based vision foundation models now provide the backbones for many contemporary 3-D perception pipelines including the ViT-based SLAM and multi-view geometry systems discussed in the next section. Their globally attentive, highly transferable features simplify camera pose, depth and scene understanding tasks that previously relied on handcrafted geometric modules.

Building on ViT backbones, recent work has begun to replace individual SLAM stages with neural-only modules: estimating the motion of up to 70k points over an extended period of time; monocular depth or full per-pixel geometry inferred in a single pass; and directly regressing NeRF-style 3D scenes from just one image [6], [26]–[28].

D. ViT SLAM

Today's state of the art has begun to treat SLAM itself as just another feed-forward vision problem. Instead of using the traditional pose graph optimization (PGO), or even chaining separate learned modules for tracking, mapping, and optimization, end-to-end ViT architectures now predict camera poses, per-view depth, and world-aligned point maps with dense tracking features in a single inference pass. Visual geometry grounded structure from motion (VGGSfM) introduced a fully differentiable SfM pipeline that could be trained end-to-end. The classical incremental pipeline of pair-wise matching, RANSAC camera registration, and bundle-adjustment is replaced with a point tracker module, a transformer camera-predictor that estimates all intrinsics and extrinsics, and a differentiable Levenberg-Marquardt layer to perform the bundle adjustment while still allowing gradients to flow through the network [7]. Pushing things further, dense and unconstrained stereo 3D reconstruction (DUSt3R) replaces the SfM chain with a pair-wise ViT that regresses a point-map for each image, which refers to a dense field of 3D points already expressed in the first view's coordinate frame. To extend from image pairs to scenes, DUSt3R drops the bundle adjustment in favor of a lightweight global alignment optimization that fits a single 3D similarity transform per image pair. Unlike VGGSfM which still need ground-truth camera intrinsics during training and relies on BA during inference, DUSt3R is trained purely on point-map regressions and infers both intrinsics and extrinsics from its own predictions [29].

The visual geometry grounded transformer (VGGT) makes the final abstraction to complete the fully feed-forward neural SLAM pipeline. Unlike DUSt3R/MASt3R, which reconcile pairwise predictions with global alignment, or VGGSfM, which still relies on differentiable bundle adjustment, VGGT removes virtually all geometry-specific post-optimization: a single forward pass simultaneously predicts calibrated cameras, per-view depth, world-aligned point maps and dense tracking features for up to hundreds of images, typically in $\leq$ 0.2s on an H100 GPU [8].

III. Method

Our initial experiments use the Visual Geometry Grounded Transformer (VGGT) [8] as a *feed-forward pose backbone*. Two different post-processing pipelines are evaluated:

1) **Sliding-window VGGT (Sim(3))**: the original script we implemented runs VGGT in overlapping temporal windows and performs a single global *similarity* (scale + rigid) alignment to ground-truth.
2) **VGGT–SLAM (SL(4))**: an extended pipeline that incrementally estimates a full projective homography between windows, augments the graph with loop–closure factors and optimizes on the SL(4) manifold, following the formulation of Murai [30].

The first approach serves as our main baseline used for on platform testing; the second one is presented as an extension based on further reading and presentation feedback.

A. Sliding-window VGGT with global scale search

Given an image sequence $\{\mathbf{I}_k\}_{k=1}^{N}$ we process it in windows $\mathcal{W}_\ell = \{k_\ell, \ldots, k_\ell + w - 1\}$ of fixed size w and stride $s = w - o$ (overlap o). For every window VGGT outputs

$$\{\mathbf{R}_k,\ \mathbf{t}_k,\ \mathbf{K}_k,\ \mathbf{D}_k,\ \mathbf{F}_k\}_{k \in \mathcal{W}_\ell}\,, \tag{1}$$

where $\mathbf{R}_k \in \mathrm{SO}(3)$, $\mathbf{t}_k \in \mathbb{R}^3$, $\mathbf{K}_k$ are per-frame intrinsics, $\mathbf{D}_k$ depth maps and $\mathbf{F}_k$ dense tracking features. Poses are expressed in the coordinate frame of the *first* image of the window; hence no additional alignment is required *inside* $\mathcal{W}_\ell$.

Across windows we originally treated VGGT as metrically ambiguous up to a single scalar $\alpha > 0$. We therefore estimate a global similarity transform $\mathbf{S} = (\alpha, \mathbf{R}, \mathbf{t}) \in \mathrm{Sim}(3)$ that minimizes the squared registration error w.r.t. OptiTrack ground truth $\{\mathbf{p}_k^{\mathrm{gt}}\}$:

$$\alpha^\star, \mathbf{R}^\star, \mathbf{t}^\star \;=\; \arg\min_{\alpha,\mathbf{R},\mathbf{t}} \sum_{k\in\mathcal{S}} \left\| \mathbf{R}\,(\alpha\,\mathbf{p}_k^{\mathrm{vggt}}) + \mathbf{t} - \mathbf{p}_k^{\mathrm{gt}} \right\|^2, \quad (2)$$

where $\mathbf{p}_k^{\mathrm{vggt}} = -\mathbf{R}_k^\top \mathbf{t}_k$ denotes the camera centre predicted by VGGT and $\mathcal{S}$ is the set of time-synchronised frames. To avoid solving a nonlinear problem we adopt a one-dimensional *grid search* over $\alpha \in [0.1, 3.0]$ with step 0.01; for each candidate scale the closed form Umeyama solution provides $(\mathbf{R}, \mathbf{t})$. The best scale is selected according to the minimal root-mean-square error (RMSE).

Finally, every pose is mapped to the world frame as

$$\mathbf{T}_k^w \;=\; \begin{bmatrix} \mathbf{R}^\star & \mathbf{t}^\star \\ \mathbf{0}^\top & 1 \end{bmatrix} \begin{bmatrix} \mathbf{R}_k & \alpha^\star\,\mathbf{t}_k \\ \mathbf{0}^\top & 1 \end{bmatrix}. \quad (3)$$

This "Sim(3) VGGT" method is entirely feed-forward, runs in ~ 0.2s per window on a single RTX-4090, and is the *primary* estimator used in all subsequent experiments.

B. Projective alignment on the SL(4) manifold

While the similarity transformation above suffices when VGGT predicts a metric scene, in practice scale drift, shearing and perspective distortions appear in low-parallax or texture-poor regions. Inspired by VGGT-SLAM [30, Sec. 4], we extend the baseline with a full 15-DoF homography alignment.

Let $\mathcal{S}_\ell$ be the submap produced for window ℓ. From every pair of consecutive submaps $(\ell-1, \ell)$ we take the w_o overlapping frames, back-project $\tau \approx 5 \times 10^4$ high-confidence depth pixels to obtain two local point clouds $\mathbf{X} \subset \mathbb{R}^3$, $\mathbf{Y} \subset \mathbb{R}^3$, and estimate the relative homography

$$\mathbf{H}_{\ell\leftarrow(\ell-1)} \;=\; \arg\min_{\mathbf{H}\in \mathrm{SL}(4)} \sum_{i=1}^{N} \left\| \mathbf{x}_i - \pi\big(\mathbf{H}\,\hat{\mathbf{y}}_i\big) \right\|^2, \quad (4)$$

with $\hat{\mathbf{y}}_i = [\mathbf{y}_i^\top\ 1]^\top$ and $\pi(\cdot)$ homogeneous division. We employ a 5-point RANSAC solver and enforce $\det \mathbf{H} = 1$ (SL(4)). The global pose of window ℓ becomes $\mathbf{H}_\ell = \mathbf{H}_{\ell\leftarrow(\ell-1)}\,\mathbf{H}_{\ell-1}$.

a) Loop closures: A CLS token $\mathbf{c}_\ell \in \mathbb{R}^{2048}$ is stored for each window. Cosine similarity retrieval (threshold 0.95) provides candidate loop pairs (i, j) that satisfy a frame gap ≥ 100. For every accepted pair we repeat the point-based alignment of (4) and left-multiply the resulting correction to all later submaps, mimicking the backend factor-graph update of [30].

b) Pose recovery: Given $\mathbf{H}_\ell$ and the local 4×4 camera matrix $\mathbf{P}_{k|\ell}$ returned by VGGT we convert to the world frame via

$$\mathbf{P}_k^w \;=\; \mathbf{H}_\ell^{-1}\,\mathbf{P}_{k|\ell}, \qquad \mathbf{T}_k^w = \mathrm{RQdecomp}\big(\mathbf{P}_k^w\big), \quad (5)$$

where $\mathrm{RQdecomp}(\cdot)$ denotes the standard projective camera decomposition into $(\mathbf{R}, \mathbf{t})$.

C. Implementation details

We use the public `facebook/VGGT-1B` checkpoint, window size $w = 25$, overlap $o = 5$ and depth-confidence threshold $\tau_{\mathrm{conf}} = 0.8$. Homographies are estimated with $5\,000$ RANSAC iterations and an inlier threshold of 2 cm; early exit is triggered once the inlier ratio exceeds 80%. All experiments were run on a single RTX-4090 GPU.

D. Real–time ROS2 Integration for On–board Tests

To verify that VGGT can operate in an *online* setting we implemented a ROS 2 node that runs directly on the companion computer of the *ModalAI Starling 2 MAX*. Compressed 1280×720 RGB frames are published by the drone's flight controller at 30 Hz on `/hires_front_small_color`. The node subscribes with a `BEST_EFFORT` QoS and converts each `sensor_msgs/Image` to a `numpy` array via `CV_BRIDGE`. All processing is performed in monotonic wall time; if no frame arrives within a user–set timeout (`image_timeout`=1 s) the sliding window is reset.

a) Sliding-window inference: Exactly w images (default $w = 5$) are cached in a deque with stride $s = 1$ ($o = w-1$). Once the window is full we pass it through VGGT as previously explain in the post-processing scripts. The local extrinsics $\{\mathbf{P}_k\}_{k=1}^w$ are aligned to the global frame by the o-frame overlap rule of Sec. III-A:

$$\mathbf{P}_k^{(G)} = \mathbf{H}_{\mathrm{win}\to G}^{-1}\,\mathbf{P}_k, \qquad k = 1, \ldots, w. \quad (6)$$

b) PX4 visual–odometry output: For every un-published frame k we form the pose $\mathbf{T}_k^w = (\mathbf{R}_k^{(G)}, \mathbf{t}_k^{(G)})$ and emit

- `/fmu/in/vehicle_visual_odometry` (PX4_MSGS/VEHICLEODOMETRY): position, orientation and timestamp in μs;
- `/vggt/pose` (GEOMETRY_MSGS/POSESTAMPED) for visualization in RVIZ2.

c) Safety and reset logic: Any runtime exception or an image–stream gap $\Delta t >$ `image_timeout` triggers a full window reset, clearing both deques and re–initialising $\mathbf{H}_{\mathrm{win}\to G} = \mathbf{I}_4$ so that stale transforms cannot contaminate the next segment.

IV. RESULTS

We preform several experiments to validate the accuracy of our methods compared to a number of baselines.

A. Baselines

1) OptiTrack: Optitrack is a standard external camera based tracking system which is common in research flight rooms. Optitrack works by using retro-reflective markers placed on a drone which are illuminated by IR emitters on a series of cameras. By identifying the locations of markers across several cameras the exact pose of the object in question can be solved. Optitrack regularly achieves accuracy in the range of 1mm over distances of +1m. In our case we consider Optitrack a ground truth signal. Typically Optitrack provides tracking at around 200hz.



Fig. 1. Picture of the Stanford flight room.

2) COLMAP: COLMAP [10] is a common structure from motion algorithm which has become standard in computer vision for resolving poses from unlabeled videos. COLMAP is a non metric algorithm which only measures relative intrinsics and extremists parameters of the cameras. In our case we preform a scale and orientation alignment using Umeyama's method [31]. COLMAP is much slower than real-time and is run after collecting the full dataset. The resulting pose estimates are generated at 5hz.

3) Visual Inertia Odometry (VIO): We utilize a ModalAI Starling 2 Max drone which also produces an online position estimation through its own integrated Qualcom VIO system. This is metric and therefore does not need to be aligned. The onboard VIO system runs at approximately 30hz.

B. Experimental Setup

We analyze the data from several flights within the Stanford flight room in the basement of Durand. We conduct two classes of flights: 1. complex flights with rotation and movement in all axis, 2. simple flights which include movement in only a single direction. A picture of the Stanford flight room is shown in Fig. 1. Notice that the wall are mostly featureless, we think this may explain some of the issues that we experience in our method please see our limitations section for more details.

We utilize a ModalAI Starling 2 Max for our experiments. Rather than collecting data in real flight situations we instead manually "fly" the drone in order to create clear images representing the best possible situation for our method.

C. Complex Flights

For our first experiment we follow a complex trajectory which involves movements and rotations in all 6 axes. For this flight we compare the performance of VGGT with all 3 of our baselines. The dataset characteristics are shown in Table. I. The quantitative results are shown in Table. II. Overall we find that VGGT preforms quite poorly compared to our baselines. Small sections of divergence greatly affect the pose estimates over the 30 second flight. Our results are shown in Fig. 2. We show both the 3D trajectories along with some of the point clouds returned by the VGGT method. Due to the poor performance of the method in this flight test we conduct a number of additional tests in which we examine more targeted regions of performance.

Diving in deeper to the results we find that the drones onboard visual odometry preforms the best, followed by COLMAP and then VGGT. We expect the onboard visual odometry to preform well as it is a tightly coupled system involving both inertial and visual data across high FPS cameras. COLMAP preforms decently, yet due to its lack of inertial data is fundamentally non-metric. Furthermore COLMAP rely on clear visual features which this room somewhat lacks. Finally VGGT significantly struggles on this dataset and is not suitable for use as an estimation algorithm. Looking deeper into the results in Fig. 2 we see that the majority of the VGGT error comes from a few areas of divergence which are highlighted in Fig. 2.

When looking at the homography based solution we see that it under-performs the scaled VGGT implementation, which we calculated to be 36% worse on average. We also see large discontinuities in the output, which indicate that some of the inter-window homographies were either poorly estimated or not calculated at all. Despite those drawbacks that lead to worse on average performance, we see that this approach, especially in the X and Z axis, is better able to capture the complexities of the trajectory. The estimation of the initial and final segments significantly improves, and we see in regions that remain continuous the trajectory matches are better, indicating promising performance for this method if the discontinuities can be addressed.

TABLE I
DATASET CHARACTERISTICS AND ACQUISITION PARAMETERS

Dataset	Total Points	Frequency (Hz)	Data Size
Optitrack	18,726	197.50	$(18726, 8)$
VIO	2,800	29.55	$(2800, 8)$
VGGT	463	4.99	$(463, 8)$
COLMAP	232	2.50	$(232, 8)$

TABLE II
ALIGNMENT ERROR STATISTICS RELATIVE TO OPTITRACK GROUND TRUTH

Method	Mean (m)	Max (m)	Min (m)	Std (m)
Visual Odometry	0.088	0.204	0.010	0.044
VGGT	0.809	1.657	0.258	0.380
COLMAP	0.317	0.908	0.147	0.100

D. Simple Flights

In order to further analyze the failures of our method we examine several simple flights and compare them to ground truth data using Optitrack. We conduct the following flights:

- **Move Forward**: In this trajectory the drone simply flying forward at a slow speed. With the NED (north east down) convention this corresponds to the drone moving forward in its X frame.
- **Move Right**: In this trajectory the drone moves toward the right, again at a slow speed.

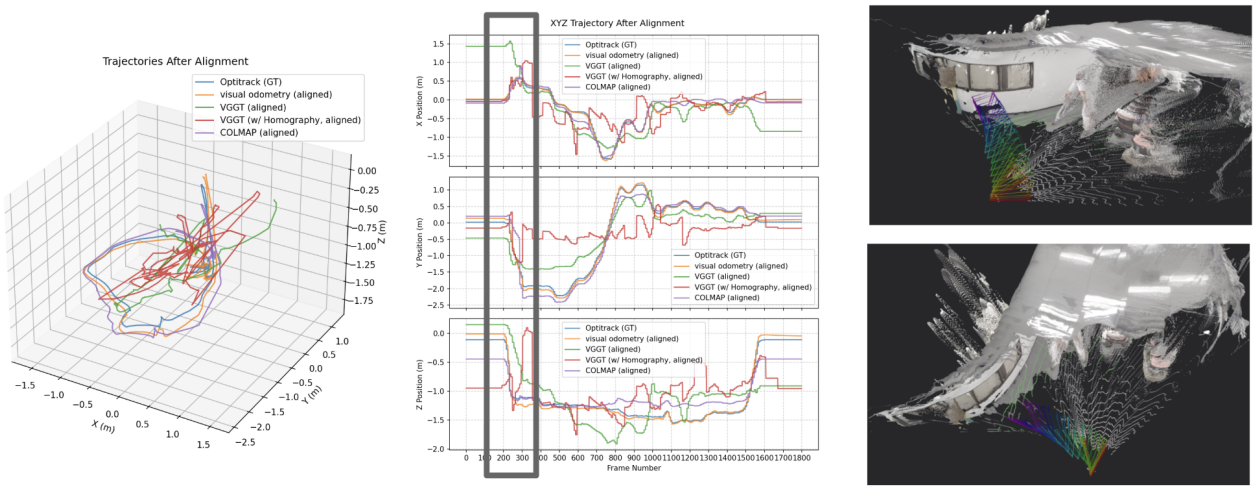


Fig. 2. Here we showcase the results of our method compared to our baselines across a complex flight scenario. On the left we show the aligned 3D trajectories. In the middle we show the resulting aligned trajectories in X, Y and Z we highlight a region in gray which shows a divergence between the GT and VGGT. On the far right we show the resulting point clouds outputted from VGGT in the highlighted region. The failure case is shown as a significant distortion of one of the flat walls in the flight room.

- **Move Right Counterclockwise**: In this trajectory the drone moves both right and rotates in a counterclockwise direction.

We show the alignment errors for each of these trajectories in Table. III. Furthermore we show the resulting trajectories for move forward and move right in Fig. 3.

For a traditional odometry system moving forward and rightward should produce similar tracking accuracies. However, due to the fully learned nature of our system we find that for our method this is not the case. In fact we find that the tracking error in *move right* significantly outperforms *move forward* by a factor of 50% less error. We conjecture that this is due to the training data used in the VGGT model. Trajectories where a camera orbits an object are much more common in internet scale datasets, and this roughly corresponds to movement in the (Y) axis in the drones frame. This explains why we see significantly better results for the move right experiment. To further examine the failure cases of our method for our third trajectory we attempt to move right while also rotating. The results for this are shown in Fig. 4.

TABLE III
TRAJECTORY PREDICTION ERRORS FOR SIMPLE FLIGHTS

Trajectory	Error (m)
Forward	.15
Right	.07
Right CounterClockwise	.20

In the *move right counter clockwise* we see significantly worse performance. We conjecture that this is again a case of out of distribution data being fed into the model. The magnitude of the error can be seen to climb as the episode elapses suggesting this error arises from a gradual rather than sudden process. We showcase the quantitative results of these experiments in Table. III.

V. LIMITATIONS

As noted above the scale-only alignment surprisingly outperforms full SL(4) chaining, but neither perform particularly well when compared against the VIO, COLMAP, or ground truth OptiTrack data. Limited visual diversity may be a contributing factor to this. Our flight room experiments revealed that large white walls and simple floor carpet may provide insufficient texture for robust feature matching. The VGGT-SLAM paper reports $> 3\times$ error when parallax is $< 5°$, which nearly matches the performance degradation we observe during purely forward motion [30]. Furthermore, the fixed sliding window approach assumes slow motion; fast yaw spins make overlap frames non-corresponding in content, causing the VGGT output to fit noise rather than true geometric relationships.

A. Failure of Homography method

One of the main possible causes of this failure could be the lack of a global bundle/factor backend. Unlike VGGT-SLAM, our SL(4) variant simply composes odometry homographies and then pre-multiplies a loop-edge correction onto *later* windows. Successive 15-DoF homographies will magnify small errors in rotation, projective scale and 4th-row parameters. There is no joint optimization that re-balances *all* sub-maps, so early windows accumulate scale and shear drift that the single post-loop homography cannot solve. Additionally, we found in testing that several loop edges retained $< 30\%$ inliers or produced near-singular $\mathbf{H}$, so without robust graph optimization these bad edges propagated directly into the global transform chain, which is manifested in the discontinuous nature of the output trajectory.

For this method it was necessary to extract point clouds from the model to run the RANSAC, unlike in the simple scaling VGGT method were we worked exclusively with

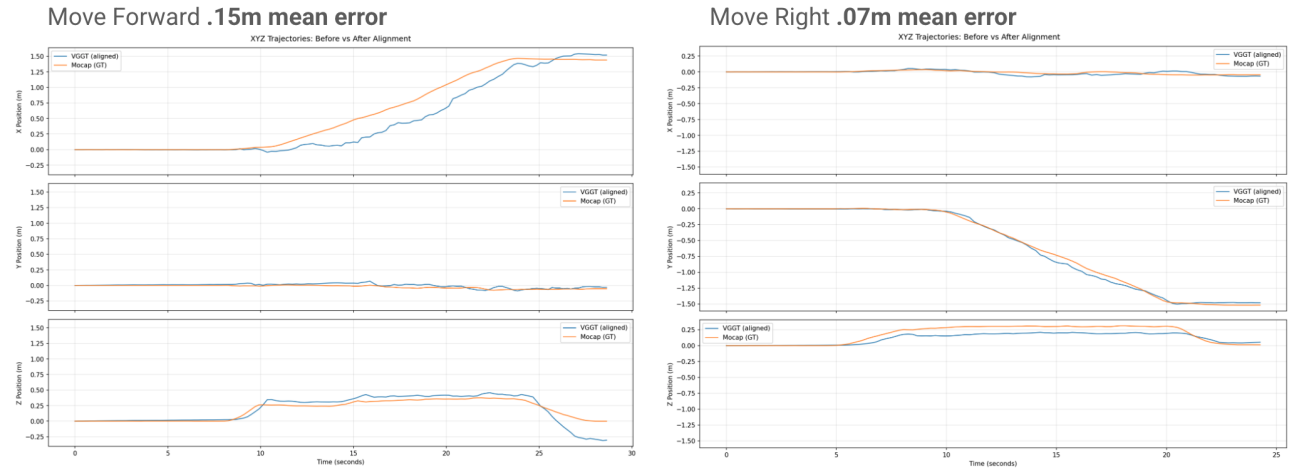


Fig. 3. Here we show the aligned trajectories produced by our method on two simple trajectories. On the left we show **move forward** and on the right we show **move right**. Notice the impressive accuracy of our method on the right compared to the relative failure on the left.

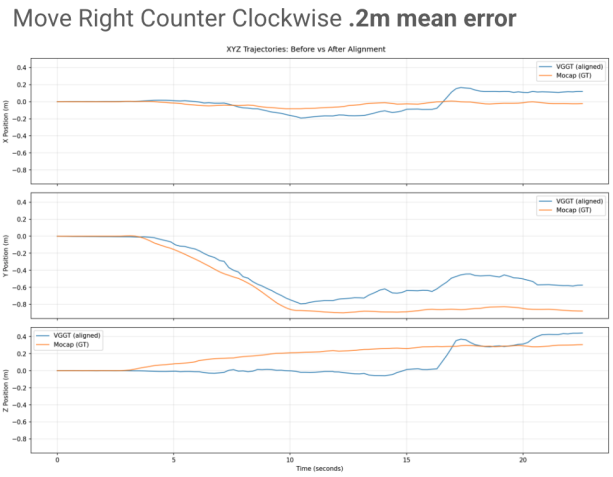


Fig. 4. Here we show the aligned trajectory produced by our method on the **move right counter-clockwise** trajectory.

the camera extrinsics. During testing, some windows lacked any correspondences despite tuning for generation of point clouds with upwards of 250,000 points. This likely occurred in regions of pure rotation as that provides minimal baseline for reliable depth estimation and correspondence matching, further exasperating the discontinuity issues.

Finally, the output of VGGT depths are metric but biased; while the global scale grid search partially corrects this bias in Sim(3), the SL(4) chain trusts VGGT's raw scale in every window, causing depth bias to manifest as translational drift.

VI. Conclusion

We set out to answer a simple question: *can a feed-forward vision foundation model replace the geometry-heavy front–end of a classical SLAM stack?* Our study provides the first systematic evaluation of VGGT in a real-robot setting and reveals three clear take-aways.

1) Feed-forward VGGT is already competitive, but only after a single Sim(3) fit. Running VGGT in 25-frame windows and aligning the output with a one-parameter global scale yields a median drift of 0.83 m over a 30 m flight. This is within a factor $8\times$ the tightly–coupled VIO and $2.5\times$ offline COLMAP without IMU. In scenes rich in sideways parallax we observe errors as low as 0.07 m (*Move Right* experiment).

2) Naive SL(4) chaining hurts more than it helps. Replacing the Sim(3) grid search by incremental 15-DoF homographies degraded mean accuracy by 36 %. Analysis shows the loss is dominated by (i) un-optimised composition of noisy homographies and (ii) one-sided loop-closure updates that fail to enforce true cycle consistency. This likely confirms the need for a full pose-graph backend as advocated in VGGT-SLAM.

3) Real-time deployment is feasible on embedded hardware. A ROS 2 node running on the Starling-2 companion computer streams 30 Hz images, executes VGGT at 4–5 Hz and publishes PX4 odometry with < 260 ms end-to-end latency. Hover tests stay within 0.15 m of OptiTrack ground truth, proving that foundation-model visual odometry can be closed on-board today.

Future work. The experiments highlight three avenues for improvement:

1) *Back-end optimization.* Integrating a lightweight SL(4) factor-graph or a learned SE(3)+scale refinement module should close the 36% gap between the homography and Sim(3) variants.
2) *Cross-modal fusion.* VGGT's drift is largely scale-bias; a Kalman-style fusion with high-rate IMU could stabilise long, low-texture segments while preserving VGGT's global consistency.
3) *Data-efficient tuning.* We observe pronounced domain shift in forward-motion flights. A short supervised fine-tune on a few thousand OptiTrack images may bring

VGGT within the performance envelope required for fully vision-only autonomous flight.

In summary, foundation pose models have reached a tipping point: with minimal post-processing they already match classical SfM in accuracy and surpass it by two orders of magnitude in speed. Coupled with principled back-end optimisation and sensor fusion, they promise a new generation of SLAM systems that are both robust and real-time, yet free of hand-engineered geometry.

REFERENCES

[1] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics (Intelligent Robotics and Autonomous Agents)*. The MIT Press, 2005.

[2] R. Mur-Artal, J. M. M. Montiel, and J. D. Tardós, "Orb-slam: A versatile and accurate monocular slam system," *IEEE Transactions on Robotics*, vol. 31, no. 5, pp. 1147–1163, 2015.

[3] A. J. Davison, I. D. Reid, N. D. Molton, and O. Stasse, "Monoslam: Real-time single camera slam," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 29, no. 6, pp. 1052–1067, 2007.

[4] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," in *Advances in Neural Information Processing Systems*, F. Pereira, C. Burges, L. Bottou, and K. Weinberger, Eds., vol. 25. Curran Associates, Inc., 2012.

[5] M. Oquab, T. Darcet, T. Moutakanni, H. Vo, M. Szafraniec, V. Khalidov, P. Fernandez, D. Haziza, F. Massa, A. El-Nouby, M. Assran, N. Ballas, W. Galuba, R. Howes, P.-Y. Huang, S.-W. Li, I. Misra, M. Rabbat, V. Sharma, G. Synnaeve, H. Xu, H. Jegou, J. Mairal, P. Labatut, A. Joulin, and P. Bojanowski, "Dinov2: Learning robust visual features without supervision," 2024. [Online]. Available: https://arxiv.org/abs/2304.07193

[6] L. Yang, B. Kang, Z. Huang, X. Xu, J. Feng, and H. Zhao, "Depth anything: Unleashing the power of large-scale unlabeled data," 2024. [Online]. Available: https://arxiv.org/abs/2401.10891

[7] J. Wang, N. Karaev, C. Rupprecht, and D. Novotny, "Visual geometry grounded deep structure from motion," 2023. [Online]. Available: https://arxiv.org/abs/2312.04563

[8] J. Wang, M. Chen, N. Karaev, A. Vedaldi, C. Rupprecht, and D. Novotny, "Vggt: Visual geometry grounded transformer," 2025. [Online]. Available: https://arxiv.org/abs/2503.11651

[9] R. Murai, E. Dexheimer, and A. J. Davison, "MASt3R-SLAM: Real-time dense SLAM with 3D reconstruction priors," *arXiv preprint*, 2024.

[10] J. L. Schönberger and J.-M. Frahm, "Structure-from-motion revisited," in *Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.

[11] J. L. Schönberger, E. Zheng, M. Pollefeys, and J.-M. Frahm, "Pixelwise view selection for unstructured multi-view stereo," in *European Conference on Computer Vision (ECCV)*, 2016.

[12] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng, "Nerf: representing scenes as neural radiance fields for view synthesis," *Commun. ACM*, vol. 65, no. 1, p. 99–106, Dec. 2021. [Online]. Available: https://doi.org/10.1145/3503250

[13] M. Adamkiewicz, T. Chen, A. Caccavale, R. Gardner, P. Culbertson, J. Bohg, and M. Schwager, "Vision-only robot navigation in a neural radiance world," *IEEE Robotics and Automation Letters*, vol. 7, no. 2, pp. 4606–4613, 2022.

[14] W. Shen, G. Yang, A. Yu, J. Wong, L. P. Kaelbling, and P. Isola, "Distilled feature fields enable few-shot language-guided manipulation," in *7th Annual Conference on Robot Learning*, 2023.

[15] B. Kerbl, G. Kopanas, T. Leimkuehler, and G. Drettakis, "3d gaussian splatting for real-time radiance field rendering," *ACM Trans. Graph.*, vol. 42, no. 4, Jul. 2023. [Online]. Available: https://doi.org/10.1145/3592433

[16] J. Yu, J. E. Low, K. Nagami, and M. Schwager, "Nerfbridge: Bringing real-time, online neural radiance field training to robotics," *arXiv preprint arXiv:2305.09761*, 2023.

[17] A. Rosinol, J. J. Leonard, and L. Carlone, "Nerf-slam: Real-time dense monocular slam with neural radiance fields," *arXiv preprint arXiv:2210.13641*, 2022.

[18] H. Li, X. Gu, W. Yuan, L. Yang, Z. Dong, and P. Tan, "Dense rgb slam with neural implicit maps," in *Proceedings of the International Conference on Learning Representations*, 2023. [Online]. Available: https://openreview.net/forum?id=QUK1ExlbbA

[19] W. Zhang, T. Sun, S. Wang, Q. Cheng, and N. Haala, "Hi-slam: Monocular real-time dense mapping with hybrid implicit fields," *IEEE Robotics and Automation Letters*, 2024.

[20] C. Yan, D. Qu, D. Xu, B. Zhao, Z. Wang, D. Wang, and X. Li, "Gs-slam: Dense visual slam with 3d gaussian splatting," in *CVPR*, 2024.

[21] H. Matsuki, R. Murai, P. H. J. Kelly, and A. J. Davison, "Gaussian Splatting SLAM," 2024.

[22] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," 2015. [Online]. Available: https://arxiv.org/abs/1409.1556

[23] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, "A simple framework for contrastive learning of visual representations," 2020. [Online]. Available: https://arxiv.org/abs/2002.05709

[24] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," 2023. [Online]. Available: https://arxiv.org/abs/1706.03762

[25] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, "An image is worth 16x16 words: Transformers for image recognition at scale," 2021. [Online]. Available: https://arxiv.org/abs/2010.11929

[26] N. Karaev, I. Rocco, B. Graham, N. Neverova, A. Vedaldi, and C. Rupprecht, "Cotracker: It is better to track together," 2024. [Online]. Available: https://arxiv.org/abs/2307.07635

[27] R. Wang, S. Xu, C. Dai, J. Xiang, Y. Deng, X. Tong, and J. Yang, "Moge: Unlocking accurate monocular geometry estimation for open-domain images with optimal training supervision," 2025. [Online]. Available: https://arxiv.org/abs/2410.19115

[28] Y. Hong, K. Zhang, J. Gu, S. Bi, Y. Zhou, D. Liu, F. Liu, K. Sunkavalli, T. Bui, and H. Tan, "Lrm: Large reconstruction model for single image to 3d," 2024. [Online]. Available: https://arxiv.org/abs/2311.04400

[29] S. Wang, V. Leroy, Y. Cabon, B. Chidlovskii, and J. Revaud, "Dust3r: Geometric 3d vision made easy," 2024. [Online]. Available: https://arxiv.org/abs/2312.14132

[30] D. Maggio, H. Lim, and L. Carlone, "Vggt-slam: Dense rgb slam optimized on the sl(4) manifold," 2025. [Online]. Available: https://arxiv.org/abs/2505.12549

[31] S. Umeyama, "Least-squares estimation of transformation parameters between two point patterns," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 13, no. 4, pp. 376–380, 1991.